

第 3 周：图像预处理与增强

试卷拍照模糊怎么办？

北京石油化工学院\人工智能研究院\王文通

北京石油化工学院
人工智能研究院

2025-2026 学年

需要的预备知识:

- **图像基础**

- 像素、通道、数据类型
- 灰度图与彩色图

- **NumPy 基础**

- 数组索引与切片
- 矩阵运算

- **OpenCV 基础**

- 图像读写 (Week 1 已学)
- 颜色空间转换

预备视频链接:

- NumPy 基础 (10 分钟入门)

- <https://www.bilibili.com/video/BV1Wy4y1h7ii>

- OpenCV 入门 (20 分钟)

- <https://www.bilibili.com/video/BV1pv4y1Q7yR>

重要提示

本周内容建立在 Week 1 基础上, 请确保已掌握图像读写操作!

本周内容:

- 图像处理流水线概述
- 图像去噪 (滤波器)
- 图像增强 (直方图、CLAHE)
- 图像二值化 (阈值方法)
- 几何变换 (仿射、透视)
- 综合实战案例

在阅卷系统中的位置:

- ① 图像采集 (Week 1 已完成)
- ② **图像预处理 (本周)**
- ③ 版面分析 (Week 4)
- ④ 选择题识别 (Week 5)
- ⑤ 判断题识别 (Week 6)
- ⑥ OCR 文字识别 (Week 7-8)

本周时间分配 (160 分钟 = 3 学时)

第 1 学时 (45 分钟):

- 00:10 理论: 流水线概述 (10min)
- 00:30 理论 + 演示: 图像去噪 (20min)
- 00:45 实践: 去噪代码实战 (15min)

第 2 学时 (45 分钟):

- 01:05 理论 + 演示: 图像增强 (20min)
- 01:30 理论 + 演示: 图像二值化 (25min)

第 3 学时 (70 分钟):

- 01:30-01:50 理论 + 演示: 几何变换 (20min)
- 01:50-02:10 实践: 综合实战 (20min)
- 02:10-02:30 互动测验 (20min)
- 02:30-02:40 总结与作业 (10min)

时间控制提示

如果进度落后, 建议跳过“挑战任务”

本周分组策略

分组原则：

- 每 4 人为一组
- 确保不同专业背景混合
- 建议包含：理工科、文科、无编程基础、有编程基础

角色分工：

角色	职责	适合
组长	统筹协调、进度管理	组织能力强的
算法实现者	实现图像处理代码	有编程基础的
参数调优者	调整滤波参数、优化效果	细心负责的
测试者	收集测试用例、报告问题	细心负责的

本周协作任务

用 OpenCV 实现试卷图像的完整预处理流程（去噪 → 增强 → 二值化）

本课程采用多屏协同教学方式：

主屏（左侧）：理论讲解

- PPT 幻灯片
- 概念和原理讲解
- 算法对比表格
- 互动测验

侧屏（右侧）：实时演示

- OpenCV 代码实时演示
- 图像处理效果展示
- 参数调整实时反馈
- 调试过程展示

移动设备互动

使用手机参与互动测验（问卷星）

代码演示程序对应关系

运行方式：在 code/文件夹下运行 `python gui_launcher.py`，选择对应的演示程序

PPT 页面	演示程序	可调参数
图像去噪	<code>gui_01_denoising.py</code>	核大小、滤波类型、Sigma
图像增强	<code>gui_02_enhancement.py</code>	CLAHE 参数、Gamma 值
图像二值化	<code>gui_03_binarization.py</code>	阈值方法、块大小、C 值
几何变换	<code>gui_04_geometry.py</code>	旋转角度、缩放、平移

提示

每个演示程序都有交互式参数滑块，拖动即可实时观察效果变化！

选择适合你的学习路径

根据你的基础和兴趣，选择适合的本周学习路径：

观察者路径

适合：无编程基础

- 理解去噪、增强的原理
- 观看代码效果演示
- 完成基础观察任务

”理解原理最重要”

使用者路径

适合：有编程基础

- 运行去噪/增强代码
- 调整滤波参数观察效果
- 完成核心实践任务

”动手实践出真知”

创造者路径

适合：编程基础较好

- 修改滤波算法参数组合
- 创新预处理流程设计
- 完成挑战优化任务

”探索与创新”

你可以随时在不同路径间切换！
本周核心目标：理解图像预处理的**完整流程**

试卷拍照的五大问题

手机拍试卷，看似简单，实则问题多多：

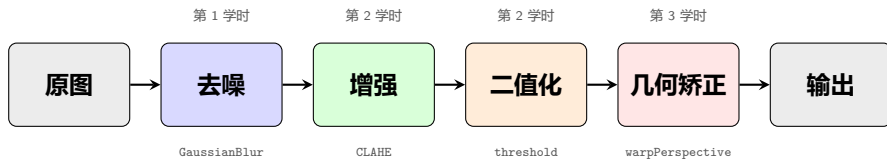
1. **模糊** ——运动模糊或失焦
手抖、距离太近导致画面不清晰
2. **光照不均** ——一侧亮一侧暗
台灯、窗户等光源造成局部过曝或欠曝
3. **倾斜** ——拍摄角度不正
试卷变成梯形，文字变形
4. **阴影遮挡** ——手指或文具挡住内容
拍摄时手、笔、橡皮留下的阴影
5. **分辨率不足** ——字迹无法辨认
距离太远、像素太少，文字变成马赛克

本周目标

解决问题 1-3：去噪、增强、几何矫正！
问题 4-5 将在后续周次中逐步解决

图像预处理流水线

一条完整的试卷图像预处理链路：



为什么是这个顺序？

- 先去噪，避免噪声干扰后续分析
- 再增强，提升有用信息的对比度
- 后二值化，将灰度图转为黑白
- 最后矫正几何畸变

关键原则

- 每一步都是为下一步**创造更好的输入**
- 流程顺序不可随意调换
- 实际项目中可根据图像质量**跳过**某些步骤

本周学习目标

学完本周，你将能够：

- ① 理解图像预处理的**完整流水线**
- ② 使用 OpenCV 实现**图像去噪**（均值/高斯/中值滤波）
- ③ 使用直方图均衡化和 CLAHE 实现**图像增强**
- ④ 掌握**二值化**方法（全局阈值 / Otsu / 自适应）
- ⑤ 完成试卷图像的**几何矫正**（仿射/透视变换）

贯穿项目

最终目标：将一张**手机拍的试卷照片**处理为清晰、端正、适合 OCR 识别的标准图像。

学习资源：

- 课堂代码演示（GUI 交互）
- code/ 文件夹下的练习脚本
- AI 辅助提问与调试

AI 辅助提示

你可以这样问 AI：

“OpenCV 中 *GaussianBlur* 和 *medianBlur* 的区别是什么？各自适合什么场景？”

试试看，你会得到非常实用的对比分析！

噪声类型与特征

噪声类型	数学模型	特征	典型场景
高斯噪声	正态分布 $N(\mu, \sigma^2)$	每个像素随机偏移, 整体发“雾”	弱光拍照、传感器热噪声
椒盐噪声	随机黑白点	孤立的纯黑/纯白像素	传输错误、AD 转换故障
泊松噪声	泊松分布 $P(\lambda)$	亮区噪声大、暗区噪声小	X 光、显微成像
周期噪声	正弦叠加	规则条纹或网格伪影	电气干扰、摩尔纹

高斯噪声数学描述

像素值服从正态分布:

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

- μ : 均值 (通常为 0)
- σ : 标准差, 控制噪声强度

椒盐噪声数学描述

像素以概率 p 被替换:

$$f(x, y) = \begin{cases} 0 & \text{概率 } p/2 \\ 255 & \text{概率 } p/2 \\ \text{原值} & \text{概率 } 1 - p \end{cases}$$

- p : 噪声密度
- 黑白点各占一半

真实场景：手机拍试卷的四大噪声来源

- ① **手机 ISO 噪声** ——光线不足时自动提高 ISO，图像颗粒感严重
- ② **JPEG 压缩伪影** ——手机默认保存 JPEG，边缘出现方块状伪影
- ③ **灰尘颗粒** ——纸面或镜头上的灰尘，表现为椒盐状斑点
- ④ **纸张纹理** ——纸张本身的纤维纹理叠加在文字上

为什么选滤波？

- 去噪是预处理流水线的**第一步**
- 不去噪，后续二值化、OCR 都受影响
- 核心矛盾：去噪 vs 保留边缘

结论

试卷预处理首选 **中值滤波**！

- 椒盐噪声（灰尘）→ 中值滤波最有效
- ISO 噪声 → 高斯滤波辅助
- 组合使用效果更佳

空间域滤波原理

核心思想：用一个小的矩阵（**滤波核/卷积核**）在图像上滑动，对邻域像素加权求和，得到新的像素值。

卷积操作步骤

- ① 将滤波核中心对准当前像素
- ② 核与覆盖区域逐元素相乘
- ③ 所有乘积求和，作为新像素值
- ④ 滑动到下一个像素，重复上述操作

数学表达：

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) \cdot f(x + s, y + t)$$

两种常见 3×3 滤波核：

均值核 (Box Filter)：

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

高斯核 ($\sigma = 1$)：

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

高斯核**中心权重大、边缘权重小**，更接近自然模糊

边界处理策略

滤波核滑到图像边缘时，核的一部分会“悬空”——需要处理边界像素。

零填充 (Zero-padding)

边界外的像素值设为 0

- 实现简单
- 边缘出现**黑色边框**
- 不推荐

复制边界 (Replicate)

用最近的边界像素值填充

- OpenCV 默认
- 边缘过渡自然
- 推荐

镜像填充 (Reflect)

以边界为对称轴镜像翻转

- 数学上最优雅
- 保持连续性
- **推荐**

```
# OpenCV 边界处理参数
cv2.GaussianBlur(img, (5,5), 0,
    borderType=cv2.BORDER_REFLECT) # 镜像填充
^^I
```

自定义卷积核：

```
import cv2
import numpy as np

img = cv2.imread('exam.jpg')

# 自定义锐化核
kernel = np.array([
    [ 0, -1,  0],
    [-1,  5, -1],
    [ 0, -1,  0]
], dtype=np.float32)

# 应用自定义核
sharpened = cv2.filter2D(img, -1, kernel)
cv2.imshow('sharpened', sharpened)
```

锐化核解释

中心 5，四周-1，总和 =1

含义：强化自身，削弱邻居 → 边缘增强

常用滤波函数速查：

```
# 1. 均值滤波（最简单）
blur = cv2.blur(img, (5, 5))

# 2. 高斯滤波（最常用）
gauss = cv2.GaussianBlur(
    img, (5, 5), 0)

# 3. 中值滤波（去椒盐噪声）
median = cv2.medianBlur(img, 5)

# 4. 双边滤波（保边缘去噪）
bilat = cv2.bilateralFilter(
    img, 9, 75, 75)
cv2.imshow('bilat', bilat)
```

注意

核大小必须是**正奇数** (3, 5, 7...)
偶数或负数会报错！

二维高斯函数：

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- σ 越大，模糊范围越广
- 核大小决定“看多远”
- $\sigma = 0$ 时 OpenCV 自动计算：
 $\sigma = 0.3 \times ((ksize - 1) \times 0.5 - 1) + 0.8$

```
# sigmaX 控制水平方向模糊程度
# sigmaY 默认等于 sigmaX
result = cv2.GaussianBlur(
    img,
    (5, 5), # 核大小
    0,      # sigmaX, 0=自动计算
)
```

核大小对效果的影响：

核大小	模糊程度	适合场景
3×3	轻微	轻度噪声
5×5	中等	试卷一般噪声
7×7	较强	重度噪声

经验法则

- 核太大 → 文字也被模糊掉
- 核太小 → 噪声去不干净
- 试卷场景推荐 (5, 5)

中值滤波详解

原理:

- 1 取当前像素周围 $k \times k$ 邻域的所有像素值
- 2 将这些值**从小到大排序**
- 3 取**中间值**作为新像素值

示例 (3×3 邻域):

$$\begin{bmatrix} 10 & 15 & 12 \\ 200 & 11 & 14 \\ 13 & 16 & 11 \end{bmatrix}$$

排序后: [10, 11, 11, 12, **13**, 14, 15, 16, 200]

中值 = **13** (200 被自动剔除)

```
# 中值滤波
result = cv2.medianBlur(img, 5)
cv2.imshow('result', result)
```

为什么适合椒盐噪声?

- 椒盐噪声是**极端值** (0 或 255)
- 排序后极端值被推到两端
- 中值不受极端值影响
- 均值滤波会被极端值拉偏!

中值 vs 高斯对比:

	高斯滤波	中值滤波
原理	加权平均	取中值
去椒盐	差	优
去高斯噪声	优	一般
保边缘	一般	好
速度	快	较慢

核大小选择

滤波方法对比总结

方法	速度	去噪效果	保边缘	适用场景
均值滤波	快	一般	差	快速预览
高斯滤波	快	好	一般	高斯噪声、轻度去噪
中值滤波	中	优 (椒盐)	好	椒盐噪声、灰尘颗粒
双边滤波	慢	好	优	需要保边缘的场景

阅卷系统推荐流程

- ① **高斯滤波**：去除 ISO 噪声
- ② **中值滤波**：去除椒盐/灰尘
- ③ **双边滤波** (进阶)：需要精细保边缘时

选择原则

- 噪声类型未知 → 先用高斯
- 明确有椒盐噪声 → 用中值
- 边缘很关键 → 用双边
- 算力有限 → 用高斯 (最快)

一句话总结

先看噪声类型 再选滤波方法

代码实战 (1/2): 去噪函数实现

GUI 演示程序: `gui_01_denoising.py`

运行 `python gui_launcher.py` 选择 “去噪演示”, 可交互调节核大小和滤波类型

实现 `compare_denoising()` 函数

```
import cv2
import numpy as np

def compare_denoising(image_path):
    """对比不同去噪方法的效果"""
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # TODO 1: 使用高斯滤波去噪
    # 提示: cv2.GaussianBlur(图像, (核大小), sigmaX)
    gauss = cv2.GaussianBlur(_____, (_____, _____), _____)

    # TODO 2: 使用中值滤波去噪
    # 提示: cv2.medianBlur(图像, 核大小)
```

代码实战 (2/2): 效果对比展示

```
import matplotlib.pyplot as plt

# 调用上一页的函数
gray, gauss, median, bilateral = compare_denoising('exam.jpg')

# TODO 1: 创建 2x2 子图布局
# 提示: plt.subplots(行数, 列数, figsize=(宽, 高))
fig, axes = plt.subplots(____, ____, figsize=(12, 10))

# TODO 2: 显示四张图
# 提示: axes[行, 列].imshow(图像, cmap='gray')
axes[0, 0].imshow(____, cmap='gray')
axes[0, 0].set_title('Original')
axes[____, ____].imshow(gauss, cmap=____)
axes[1, 0].set_title('Gaussian')
axes[1, 1].imshow(____, cmap='gray')
axes[1, 1].set_title('Median')
axes[0, 1].imshow(____, cmap='gray')
axes[0, 1].set_title('Bilateral')

# TODO 3: 去掉坐标轴
# 提示: 使用 ax.axis('off')
for ax in axes.flat:
    ax._____

plt.tight_layout()
plt.show()
"""
```

预期结果

代码找茬：这张代码有几个 Bug？

任务：以下代码有 **3 个错误**，找出并修正！

有 Bug 的去噪代码 – 请找出 3 个错误

```
import cv2

def denoise_exam(path):
    """试卷去噪函数"""
    img = cv2.imread(path)

    # Bug 1: 缺少什么关键步骤?
    # 去噪前应该先做什么?

    # Bug 2: 核大小参数有问题
    gauss = cv2.GaussianBlur(img, (4, 4), 0)

    # Bug 3: 滤波方法选错了
    # 试卷上有灰尘(椒盐噪声), 该用什么滤波?
    clean = cv2.GaussianBlur(gauss, (5, 5), 0)

    return clean

result = denoise_exam('exam.jpg')
cv2.imwrite('clean.jpg', result)
^^I
```

找 Bug 提示

讨论 (2 分钟)

去噪前忘记转换颜色空间

和同桌讨论 写出修正后的代码

为什么要增强图像？

低质量图像问题：

- **对比度低**（灰蒙蒙一片）
- 亮部过曝或暗部欠曝
- 细节不清晰
- 试卷拍照偏暗/偏亮

增强目标

- 提升对比度
- 恢复细节
- 改善视觉效果
- **方便后续处理（OCR、边缘检测）**

重要概念

图像增强 $\neq$ 去噪

去噪是“清理垃圾”，增强是“优化显示”

图像直方图

定义：直方图表示图像中各像素值级别的分布情况

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 读取灰度图
img = cv2.imread('exam.jpg', 0)

# 计算直方图
hist = cv2.calcHist([
    [img],          # 图像
    [0],           # 通道
    None,          # 掩码
    [256],         # 像素值范围
    [0, 256]       # 范围
])

# 绘制直方图
plt.figure(figsize=(8, 4))
plt.plot(hist, color='black', linewidth=1.5)
plt.xlabel('像素值 (0-255)')
plt.ylabel('像素数量')
plt.title('图像直方图')
plt.xlim([0, 256])
plt.grid(True, alpha=0.3)
plt.show()
```

直方图解读

- **左偏**：图像偏暗
- **右偏**：图像偏亮
- **集中**：对比度低（灰蒙蒙）
- **分散**：对比度高

GUI 演示程序：gui_02_enhancement.py

运行 python gui_launcher.py 选择“增强演示”，可交互调节 CLAHE 参数和 Gamma 值

直方图均衡化

原理：重新分布像素值，使直方图更均匀，增强对比度

算法步骤

- ① 统计各像素值出现次数
- ② 计算累计分布函数 (CDF)
- ③ 建立映射： $s = T(r) = (L - 1) \cdot CDF(r)$
- ④ 应用映射到所有像素

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# 读取灰度图
gray = cv2.imread('exam.jpg', 0)

# 全局直方图均衡化
equalized = cv2.equalizeHist(gray)

# 显示对比效果
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

axes[0,0].imshow(gray, cmap='gray')
axes[0,0].set_title('原图')
axes[0,0].axis('off')

axes[0,1].plot(cv2.calcHist([gray], [0], None,
                           [256], [0, 256]))
axes[0,1].set_title('原图直方图')

axes[1,0].imshow(equalized, cmap='gray')
axes[1,0].set_title('均衡化后')
axes[1,0].axis('off')
```

CLAHE (对比度受限自适应直方图均衡化)

问题：全局均衡化会导致噪声放大

CLAHE 特点

- **局部处理**：将图像分成小块
- 对每个小块独立均衡化
- **限制对比度**：防止噪声放大
- 双线性插值：消除块效应

```
import cv2
import numpy as np

# 读取灰度图
gray = cv2.imread('exam.jpg', 0)

# 创建 CLAHE 对象
clahe = cv2.createCLAHE(
    clipLimit=2.0,      # 对比度限制阈值
    tileGridSize=(8, 8) # 分块大小
)

# 应用 CLAHE
enhanced = clahe.apply(gray)

# 对比
cv2.imshow('Original', gray)
cv2.imshow('CLAHE', enhanced)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Gamma 校正

公式: $I_{out} = 255 \cdot (I_{in}/255)^\gamma$

效果解读

- $\gamma < 1$: 提亮暗部 (校正偏暗图像)
- $\gamma = 1$: 无变化
- $\gamma > 1$: 压暗亮部 (校正过曝图像)

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def gamma_correction(img, gamma):
    """Gamma 校正"""
    # 归一化到 [0, 1]
    normalized = img / 255.0
    # 应用 Gamma 变换
    corrected = np.power(normalized, gamma)
    # 恢复到 [0, 255]
    return np.uint8(corrected * 255)

# 读取图像
gray = cv2.imread('exam.jpg', 0)

# 不同 Gamma 值
gammas = [0.5, 1.0, 2.0]
results = [gamma_correction(gray, g) for g in gammas]

# 显示效果
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
for i, (g, res) in enumerate(zip(gammas, results)):
```

图像锐化

目的：增强边缘，恢复细节

方法一：拉普拉斯算子

- 二阶导数
- 突出灰度突变区域
- `cv2.Laplacian()`

方法二：Unsharp Masking

- ① 模糊原图
- ② 计算模糊图与原图之差
- ③ 将差值加回原图

```
import cv2
import numpy as np

gray = cv2.imread('exam.jpg', 0)

# 方法1: 拉普拉斯锐化
laplace = cv2.Laplacian(gray, cv2.CV_64F)
sharp1 = cv2.convertScaleAbs(laplace)

# 方法2: Unsharp Masking (更常用)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
mask = cv2.subtract(gray, blur)
sharp2 = cv2.add(gray, mask)

# OpenCV 内置方法
kernel = np.array([[ -1, -1, -1],
                   [ -1,  9, -1],
                   [ -1, -1, -1]])
sharp3 = cv2.filter2D(gray, -1, kernel)

cv2.imshow('Sharpened', sharp3)
cv2.waitKey(0)
```

增强方法对比表

方法	原理	适用场景	注意
equalizeHist	全局均衡化	整体偏暗/偏亮	噪声放大
CLAHE	局部自适应	光照不均	参数调优
Gamma 校正	幂律变换	偏暗/偏亮图像	选择合适 γ
锐化	边缘增强	细节模糊	适度使用

实战建议

- 试卷图像：先用 CLAHE 增强对比度
- 偏暗图像：尝试 $\gamma = 0.6 \sim 0.8$
- 模糊图像：适当锐化
- **组合使用**效果更好

代码实战：CLAHE + Gamma 完整流程

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

def enhance_image(image_path, gamma=0.7, clip_limit=2.0):
    """
    图像增强完整流程
    TODO: 实现以下步骤
    """
    # Step 1: 读取图像
    gray = cv2.imread(image_path, 0)

    # Step 2: CLAHE 增强对比度
    clahe = cv2.createCLAHE(clipLimit=clip_limit, tileGridSize=(8, 8))
    enhanced = clahe.apply(gray)

    # Step 3: Gamma 校正
    normalized = enhanced / 255.0
    corrected = np.power(normalized, gamma)
    final = np.uint8(corrected * 255)

    return final

# 测试
if __name__ == '__main__':
    result = enhance_image('exam.jpg')
```

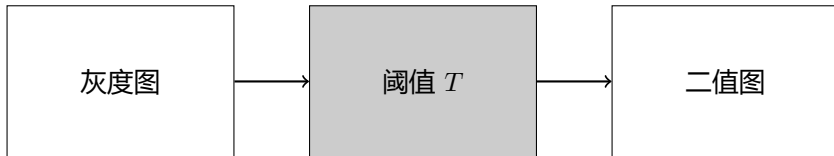
什么是二值化?

定义

- 将灰度图像转换为只有**黑白**两种颜色
- 像素值: 0 (黑) 或 255 (白)
- 公式:
$$B(x, y) = \begin{cases} 0 & I(x, y) < T \\ 255 & I(x, y) \geq T \end{cases}$$

为什么重要?

- 简化图像, 便于分析
- **OCR 识别前必经步骤**
- 边缘检测基础
- 文档扫描核心



典型应用

- 1 **OCR 文字识别**: 将文字与背景分离
- 2 文档扫描: 去除彩色背景
- 3 边缘检测: 提取轮廓
- 4 血细胞计数: 分离细胞
- 5 车牌识别: 定位车牌

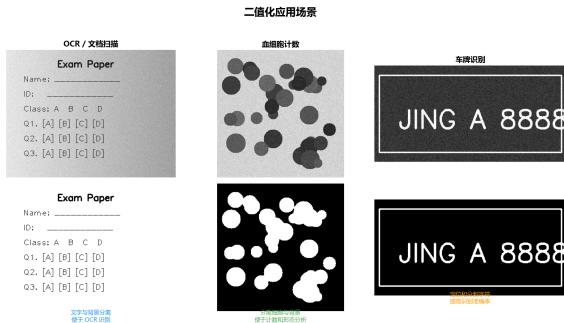


图: 三种典型场景的二值化效果

全局阈值法

原理：使用固定阈值 T 进行分割

```
import cv2
import numpy as np

# 读取灰度图
gray = cv2.imread('exam.jpg', 0)

# 方法1: 固定阈值 127
_, binary1 = cv2.threshold(
    gray, 127, 255, cv2.THRESH_BINARY
)

# 方法2: 反转二值化
_, binary2 = cv2.threshold(
    gray, 127, 255, cv2.THRESH_BINARY_INV
)

# 方法3: 截断
_, binary3 = cv2.threshold(
    gray, 127, 255, cv2.THRESH_TRUNC
)

# 方法4: 零化
_, binary4 = cv2.threshold(
    gray, 127, 255, cv2.THRESH_TOZERO
)
```

阈值类型

- THRESH_BINARY: 大于 T 设为 255
- THRESH_BINARY_INV: 反转
- THRESH_TRUNC: 截断到 T
- THRESH_TOZERO: 小于 T 设为 0

问题

固定阈值需要手动尝试，不适合光照不均图像

Otsu 自适应阈值

原理：自动寻找最佳阈值（直方图双峰时效果好）

算法思想

- ① 假设直方图呈双峰分布
- ② 寻找两峰之间的谷底作为阈值
- ③ 最大化类间方差 (Otsu 方法)

```
import cv2
import numpy as np

gray = cv2.imread('document.jpg', 0)

# Otsu 方法自动找阈值
_, otsu = cv2.threshold(
    gray,          # 原图
    0,             # 阈值（设为0，Otsu自动计算）
    255,           # 最大值
    cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

# 对比效果
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(gray, cmap='gray')
axes[0].set_title('原图')

axes[1].imshow(cv2.threshold(gray, 127, 255, cv2.
    THRESH_BINARY)[1], cmap='gray')
axes[1].set_title('固定阈值127')

axes[2].imshow(otsu, cmap='gray')
axes[2].set_title('Otsu 自适应')
```

自适应阈值

原理：根据邻域计算局部阈值

两种方法

- ADAPTIVE_THRESH_MEAN: 邻域均值
- ADAPTIVE_THRESH_GAUSSIAN: 高斯加权

参数

- blockSize: 邻域大小 (奇数)
- C: 从均值中减去的常数

```
import cv2

gray = cv2.imread('exam.jpg', 0)

# 方法1: 基于均值
adaptive_mean = cv2.adaptiveThreshold(
    gray, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY,
    blockSize=11,
    C=2
)

# 方法2: 基于高斯 (更常用)
adaptive_gaussian = cv2.adaptiveThreshold(
    gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    blockSize=11,
    C=2
)

# 参数调优建议
# - blockSize 越大, 平滑效果越强
```

二值化方法对比

方法	原理	适用场景	缺点
全局固定阈值	固定值 $T = 127$	背景均匀	不抗光照变化
Otsu 自适应	最大化类间方差 局部邻域计算	文档、书页 光照不均	对噪声敏感 可能有块效应

实战建议

- 文档扫描：Otsu 或自适应阈值
- 试卷答题卡：自适应阈值 + 后处理
- **组合使用：中值滤波去噪 + 自适应阈值**

代码实战：二值化函数

```
import cv2
import numpy as np

def binarize_image(image_path, method='otsu', blockSize=11, C=2):
    """
    二值化函数

    Args:
        image_path: 图像路径
        method: 'otsu', 'adaptive_mean', 'adaptive_gaussian'
        blockSize: 自适应阈值块大小
        C: 阈值偏移量

    Returns:
        二值化图像
    """
    # 读取灰度图
    gray = cv2.imread(image_path, 0)
    if gray is None:
        raise ValueError(f"无法读取图像: {image_path}")

    # 根据方法选择
    if method == 'otsu':
        _, binary = cv2.threshold(
            gray, 0, 255,
            cv2.THRESH_BINARY + cv2.THRESH_OTSU
```

代码找茬：找出 BUG

问题：下面的代码有什么问题？

```
import cv2
import numpy as np

def buggy_binarize(img):
    """有BUG的二值化函数"""

    # BUG 1: 忘记转灰度
    result = cv2.threshold(img, 127, 255, cv2.THRESH_BINARY)
    return result # 返回的是tuple, 不是图像!

def correct_binarize(img):
    """正确的写法"""
    # 确保是灰度图
    if len(img.shape) == 3:
        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    else:
        gray = img

    # 正确处理返回值
    _, result = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)
    return result

# 测试
```

仿射变换基础

定义

- 保持**平行性**的变换
- 直线变换后仍是直线
- 平行线保持平行

包含操作

- 平移 (Translation)
- 旋转 (Rotation)
- 缩放 (Scale)
- 倾斜 (Shear)

变换矩阵

$$M = \begin{bmatrix} a_{11} & a_{12} & b_x \\ a_{21} & a_{22} & b_y \end{bmatrix}$$

- 2×3 矩阵
- 6 个参数确定
- 用 `cv2.warpAffine()`

重要特性

仿射变换只需要 **3 个点**确定 (6 个方程)

仿射变换代码

```
import cv2
import numpy as np

# 读取图像
img = cv2.imread('exam.jpg')
h, w = img.shape[:2]

# 1. 平移变换
M_trans = np.float32([
    [1, 0, 50],      # x 方向平移 50
    [0, 1, 30]       # y 方向平移 30
])
translated = cv2.warpAffine(img, M_trans, (w, h))

# 2. 旋转变换
center = (w // 2, h // 2)
angle = 45
scale = 1.0
M_rot = cv2.getRotationMatrix2D(center, angle,
    scale)
rotated = cv2.warpAffine(img, M_rot, (w, h))

# 3. 组合变换
M = np.float32([
    [1.2, 0.2, 0],
    [0, 1.1, 0]
```

常用函数

- `cv2.warpAffine()`: 应用变换
- `cv2.getRotationMatrix2D()`: 获取旋转矩阵
- `cv2.getAffineTransform()`: 从 3 点计算矩阵

TODO 任务

实现一个函数:
输入图像 + 角度, 返回旋转后的图像

透视变换

原理： 4 点对应变换，可以矫正倾斜的矩形

变换矩阵

- 需要 **4 个点** 确定
- 3×3 变换矩阵
- 用 `cv2.getPerspectiveTransform()`
- 用 `cv2.warpPerspective()`

```
import cv2
import numpy as np

# 原图上的4个点（顺时针）
pts1 = np.float32([
    [100, 50],      # 左上
    [400, 50],      # 右上
    [400, 300],     # 右下
    [100, 300]      # 左下
])

# 希望变换到的位置
pts2 = np.float32([
    [0, 0],
    [300, 0],
    [300, 250],
    [0, 250]
])

# 计算透视变换矩阵
M = cv2.getPerspectiveTransform(pts1, pts2)

# 应用变换
h, w = 300, 300
```

自动矫正实战：试卷倾斜矫正

GUI 演示程序: `gui_04_geometry.py`

运行 `python gui_launcher.py` 选择“几何变换演示”，可交互调节旋转角度、缩放和平移参数

```
import cv2
import numpy as np

def correct_perspective(image_path):
    """
    试卷倾斜矫正完整流程
    """
    # Step 1: 读取图像
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Step 2: 边缘检测
    edges = cv2.Canny(gray, 50, 150, apertureSize=3)

    # Step 3: 霍夫变换检测直线
    lines = cv2.HoughLinesP(edges, 1, np.pi/180, 100,
                             minLineLength=100, maxLineGap=10)

    # Step 4: 计算角度
    angles = []
    for line in lines:
```

挑战任务：智能裁剪答题卡

任务描述

实现一个函数，输入试卷图片，自动找到答题卡区域并裁剪

- 1 检测最大轮廓（可能是答题卡）
- 2 获取 4 个角点
- 3 透视变换裁剪
- 4 返回裁剪后的图像

```
import cv2
import numpy as np

def crop_answer_card(image_path):
    """智能裁剪答题卡"""
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # TODO Step 1: 二值化
    # ...

    # TODO Step 2: 找轮廓
    # contours, _ = cv2.findContours(...)

    # TODO Step 3: 找最大四边形
    # hint: cv2.approxPolyDP

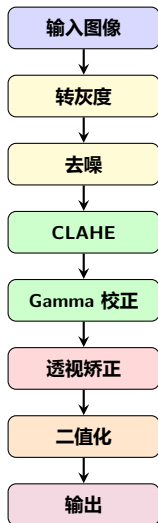
    # TODO Step 4: 透视变换裁剪
    # ...

    return cropped

# 测试
result = crop_answer_card('exam.jpg')
cv2.imshow('Cropped', result)
cv2.waitKey(0)
```

试卷预处理完整流程

处理流水线



```
import cv2
import numpy as np

def preprocess_exam(image_path):
    """试卷预处理完整流程"""
    img = cv2.imread(image_path)
    if img is None:
        raise ValueError(f"无法读取: {image_path}")

    # Step 1: 转灰度
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Step 2: 去噪
    denoised = cv2.fastNlMeansDenoising(
        gray, None, 10, 7, 21)

    # Step 3: CLAHE 对比度增强
    clahe = cv2.createCLAHE(
        clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(denoised)

    # Step 4: Gamma 校正
    gamma = 0.7
    norm = enhanced / 255.0
    corrected = np.uint8(
        np.power(norm, gamma) * 255)

    # Step 5: 透视矫正 (需提供角点)
    # M = cv2.getPerspectiveTransform(pts1, pts2)
    # warped = cv2.warpPerspective(corrected, M, (w, h))

    # Step 6: 自适应二值化
    binary = cv2.adaptiveThreshold(
```

行业应用案例

医疗影像

- X 光片增强
- CT/MRI 对比度调整
- 病灶区域增强
- 常用方法：CLAHE、Gamma

文档扫描

- 手机扫描 App
- 书籍数字化
- 名片识别
- 常用方法：透视变换、二值化

工业检测

- PCB 缺陷检测
- 产品质量检查
- 表面划痕检测
- 常用方法：锐化、边缘检测

发展趋势

- AI 驱动的智能预处理
- 端到端优化
- 自适应参数

快速问答环节 - 问题

问题 1: 高斯滤波的主要作用是?

- A 锐化图像
- B 去除高斯噪声
- C 增加对比度
- D 边缘检测

问题 2: CLAHE 中的“CLA” 指什么?

- A Central Local Adaptive
- B Contrast Limited Adaptive Histogram Equalization
- C Classical Linear Algorithm
- D Color Level Adjustment

问题 3: 自适应阈值适用于什么场景?

- A 光照均匀的图像
- B 光照不均的图像
- C 彩色图像
- D 视频帧

问题 4: warpPerspective 需要几个对应点?

- A 2 个
- B 3 个
- C 4 个
- D 任意数量

请思考答案，下一页揭晓!

快速问答环节 - 答案

问题 1: 高斯滤波的主要作用是?

- A 锐化图像
- B **去除高斯噪声** (正确)
- C 增加对比度
- D 边缘检测

问题 2: CLAHE 中的“CLA” 指什么?

- A Central Local Adaptive
- B **Contrast Limited Adaptive Histogram Equalization** (正确)
- C Classical Linear Algorithm
- D Color Level Adjustment

问题 3: 自适应阈值适用于什么场景?

- A 光照均匀的图像
- B **光照不均的图像** (正确)
- C 彩色图像
- D 视频帧

问题 4: warpPerspective 需要几个对应点?

- A 2 个
- B 3 个
- C **4 个** (正确)
- D 任意数量

恭喜! 全部正确!

代码找错挑战：二值化

找出以下 3 段代码中的错误：

代码段 1

```
# 高斯模糊
blur = cv2.GaussianBlur(
    gray, (5, 5), 0)

# 全局阈值
ret, binary = cv2.threshold(
    blur, 127, 255,
    cv2.THRESH_BINARY)
```

代码段 2

```
# 自适应阈值
binary = cv2.adaptiveThreshold(
    gray, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY, 11, 2)

# 错误: gray是彩色图
```

代码段 3

```
# Otsu 自动阈值
ret, binary = cv2.threshold(
    gray, 0, 255,
    cv2.THRESH_BINARY +
    cv2.THRESH_OTSU)

# 错误: 未先去噪
```

思考提示

- 代码段 1: 错误在缺少**灰度化步骤**
- 代码段 2: **输入必须是灰度图**, 不能用彩色图
- 代码段 3: **Otsu 前应先去噪**, 否则噪声会被放大

错误分析

❶ 代码段 1:

- 错误: 高斯模糊核大小为 (5, 5), **这是正确的**
- 但缺少灰度化步骤: `gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`

❷ 代码段 2:

- **致命错误: 输入是彩色图**
- 自适应阈值只能处理单通道
- 报错: `error: (-215:Assertion failed) src.type() == CV_8UC1`

❸ 代码段 3:

- Otsu 自动阈值对噪声**非常敏感**
- 应该在阈值前先高斯去噪
- 否则会将噪声点误判为前景

修正后的代码

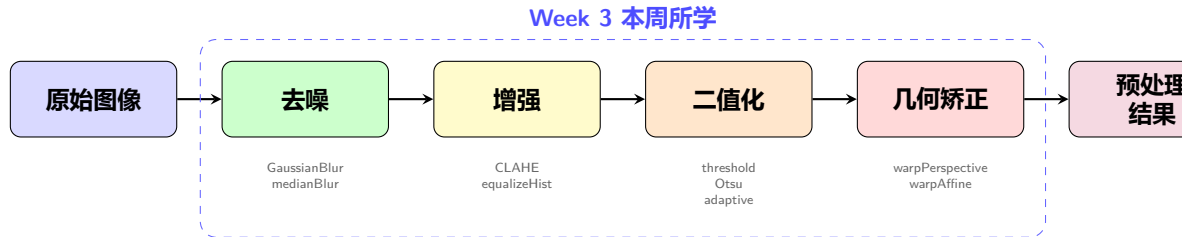
```
# 正确流程
gray = cv2.cvtColor(
    img, cv2.COLOR_BGR2GRAY)

# 1. 去噪
blur = cv2.GaussianBlur(
    gray, (5, 5), 0)

# 2. Otsu 自动阈值
ret, binary = cv2.threshold(
    blur, 0, 255,
    cv2.THRESH_BINARY +
    cv2.THRESH_OTSU)

# 或者使用自适应阈值
binary = cv2.adaptiveThreshold(
    blur, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY, 11, 2)
```

本周知识体系：图像预处理流水线



核心思想

预处理的目标：将“脏”的原始试卷图像 → “干净”的标准图像，为后续 OCR 和版面分析做准备。

核心函数回顾：本周学到哪些 API?

去噪函数

```
# 高斯滤波
blur = cv2.GaussianBlur(
    gray, (5, 5), 0)

# 中值滤波 (去椒盐噪声)
median = cv2.medianBlur(gray, 5)
```

增强函数

```
# CLAHE自适应对比度增强
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8,8))
enhanced = clahe.apply(gray)
```

二值化函数

```
# 全局阈值
ret, binary = cv2.threshold(
    blur, 127, 255,
    cv2.THRESH_BINARY)

# Otsu自动阈值
ret, binary = cv2.threshold(
    blur, 0, 255,
    cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 自适应阈值
binary = cv2.adaptiveThreshold(
    blur, 255,
    cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY, 11, 2)
```

几何矫正函数

```
# 获取变换矩阵
M = cv2.getPerspectiveTransform(
    src_pts, dst_pts)

# 透视变换
result = cv2.warpPerspective(
```

课后作业：完整试卷预处理流水线

作业要求

实现一个完整的试卷预处理流水线，处理一张倾斜、模糊、光照不均的试卷照片。

处理流程（必须包含）：

- ❶ **去噪**：使用 GaussianBlur 或 medianBlur
- ❷ **增强**：使用 CLAHE 对抗光照不均
- ❸ **二值化**：使用自适应阈值或 Otsu
- ❹ **几何矫正**：使用 warpPerspective 矫正倾斜

提交要求：

- 完整代码（.py 文件）
- 处理前后对比图
- 简要说明每个步骤的作用

参考框架

```
import cv2
import numpy as np

def preprocess_exam(img):
    # 1. 灰度化
    gray = cv2.cvtColor(
        img, cv2.COLOR_BGR2GRAY)

    # 2. 去噪
    blur = cv2.GaussianBlur(
        gray, (5, 5), 0)

    # 3. 增强
    clahe = cv2.createCLAHE(2.0)
    enhanced = clahe.apply(blur)

    # 4. 二值化
    binary = cv2.adaptiveThreshold(
        enhanced, 255,
        cv2.ADAPTIVE_THRESH_MEAN_C,
        cv2.THRESH_BINARY, 11, 2)

    # 5. 几何矫正（需要4个角点）
```

本周核心知识点:

- **去噪**: 高斯滤波、中值滤波
- **对比度增强**: CLAHE 自适应直方图均衡
- **二值化**: 全局阈值、自适应阈值、Otsu
- **几何矫正**: 透视变换 warpPerspective
- 完整预处理流水线

下周预告 (Week 4): 试卷版面分析

- **边缘检测**: Canny 算子
- **轮廓查找**: findContours
- **轮廓近似**: approxPolyDP
- **定位答题卡**: 利用轮廓定位答题区域

跨周链接

- Week 1: 图像基础
- Week 2: AI 辅助编程
- Week 3: 图像预处理 ✓
- Week 4: 版面分析 ▷
- Week 5: 选择题识别

重点提示

下周我们将进入**版面分析**, 学会如何从整张试卷图像中**自动定位**出答题卡的位置!

本周总结

核心收获

- 学会了图像预处理的完整流水线
- 掌握了去噪、增强、二值化、几何矫正四大技术
- 能够处理实际中的“脏”试卷图像